Vectorizing the Company: Leveraging Multimodal Data for Process Mining

Yorck Zisgen[1][0000−0002−9646−2829],
Matthias Pohl[2][0009−0009−7085−9539], and
Agnes Koschmider[1][0000−0001−8206−7636]

[1] University of Bayreuth, Bayreuth, Germany
[2] ames wiring GmbH, Ursensollen, Germany

Abstract. Process mining is a set of techniques that allow insights into business processes from event data recorded from information systems. The emergence of Generative AI and Large Language Models (LLM) in particular give the potential of tapping into previously unused data for process mining like in non-textual formats such as images, PowerPoint presentations, or Excel files. Leveraging this data could significantly enrich process discovery, conformance checking, and process enhancement by providing additional context and explanations. In this paper, we propose VeCo, an open-source Python library that encapsulates and streamlines the entire workflow of connecting a vector database to a local LLM and automating the vectorization of multimodal data across diverse formats, allowing LLMs to efficiently access and utilize this information for downstream tasks. We demonstrate how the added vectorized data from companies helps improving process mining results by means of two use cases.

Keywords: Generative AI · Large Language Model · Embedding · Multimodal Data · Vector Database · Process Mining.

1 Introduction

Process mining has proven to uncover valuable insights from data and has received widespread use in domains such as healthcare, production, and public administration. Since the release of GPT-3 by OpenAI, Generative Artificial Intelligence (GenAI) and particularly Large Language Models (LLMs) have received significant attention. LLMs are already diversely exploited in process mining, e.g., to discover process models from textual descriptions [6, 9, 10] or to serve as a natural language interface for process mining tasks, developing prompting strategies, and providing tool support [1, 3, 8]. Various methods for incorporating context knowledge in terms of few-shot prompting, knowledge injection, and domain expertise have consistently demonstrated a positive impact on LLM performance in process mining tasks [7, 11]. However, the application of LLMs in process mining typically focuses on processing text data and rarely uses multimodal approaches. Also, no tool exists for handling multimodal data (e.g., .pdf,

Pre-print copy of the paper accepted for presentation at the Process Mining Workshops (ICPM 2025 Int. Workshops), to appear in the Springer LNBIP series on Springer Link at https://link.springer.com/

.ppt, or .mp3) for process mining. Such a tool would need to embed content for persistent storage, create a vector database, connect to an LLM to provide contextual information, and thereby enhance process mining results.

This paper proposes VeCo[3], an open-source Python library that handles multimodal data, offers services to embed data, provides a vector database in which data is stored, and connects to local LLMs via the Ollama API. Fig. 1 illustrates our contribution. Company domain knowledge is documented in a range of file formats. VeCo offers an import interface, decides internally how to break down multimodality, and extracts knowledge by transforming modalities into text, depending on the encountered file format. It embeds the extracted texts, optionally compressing these, stores them in a JSON vector database, and connects to the Ollama[4] API to provide the domain knowledge to a third-party local LLM[5].

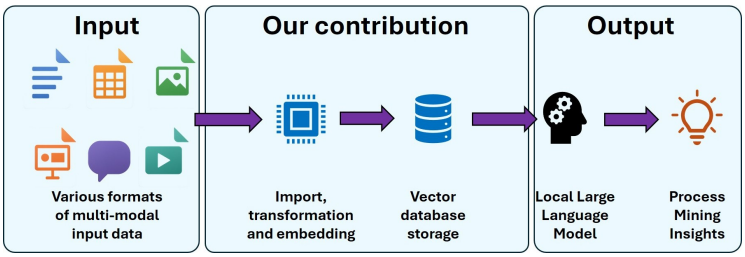

Fig. 1. The VeCo pipeline - accessing multimodal data for process mining

We demonstrate the benefit of leveraging domain knowledge for LLM-based process mining tasks on two use cases: In use case 1 (cf. Sec. 4.1), we show how an LLM relies on domain knowledge to help identify relevant data and prevent the production of an incomplete event log and thereby discover an incomplete and meaningless process model. In use case 2 (cf. Sec. 4.2), we found that conformance checking can be unreliable when using outdated or incomplete process models. Keeping track of changes to these models through domain knowledge helps to avoid *'false positives'*, and therefore draw the right conclusions (update the de-jure model instead of re-train employees) from a perceived model skip. Apart from these two use cases, additional scenarios where domain knowledge can prove helpful for process mining are possible, such as identifying the correct notion of a case ID or appropriate activity labeling for process discovery, extracting decision rules or responsibilities for activities in process enhancement, or identifying the proper business objects in object-centric process mining.
The remainder of the paper is structured as follows. We discuss related work in Sec. 2. In Sec. 3 we introduce our approach, before demonstrating it on two use cases in Sec. 4. Sec. 5 concludes this paper.

[3] All code and data are available at https://github.com/MatthiasPohlAmberg/VeCo
[4] Ollama https://ollama.com/
[5] Local LLM models https://ollama.com/search

2 Related Work

Related works include those that *i)* extract business process insights from natural language textual descriptions, *ii)* leverage domain knowledge to enhance process mining results and *iii)* use LLMs for natural language interaction with processes and data.

The extraction of business process models from textual descriptions using LLMs is the predominant use case, and plenty of related approaches exist ([6, 9, 10]). Daclin et. al [6] implemented an approach that uses speech-to-text conversion that translates an interview into text, and then uses GPT-4 to extract a BPMN model from the text. Klievtsova et al. [9] suggested Conversational Process Modeling (CPM), which involves a multi-step dialogue with the LLM to iteratively construct a business process model. [9] concludes that current LLMs are generally reliable. They acknowledge that while chatbots are generally 'ready as-is', a domain expert should anyway check the results. Kourani et al. [10] propose a framework that utilizes LLMs to automate the generation of process models. They use prompt engineering strategies such as knowledge injection and an iterative refinement loop to increase the quality of the results. Notably, while their focus is on the performance of the proposed framework, technically, their approach of knowledge injection is related to our approach.

Dixit et al. evaluate the impact of incorporating domain knowledge to improve process model discovery [7]. Their approach first applies process discovery and then asks domain experts for declare constraints. They propose an algorithm that generates a Pareto set of process trees from the event log satisfying the constraints while balancing for fitness, precision, simplicity, and generalization. In [11], Nourizafar et al. integrate domain knowledge into the process discovery step by using an LLM to convert textual expert knowledge into a set of declarative rules, to which a discovered process model has to conform.

van der Aalst [1] argues that GenAI will simplify extracting event data by efficiently processing queries and questions in natural language. Furthermore, while GenAI might not be able to replace core process mining algorithms, it could help explain process mining results. He also acknowledges that a human user will be needed to verify the correctness of a GenAI response. Jessen et al. [8] utilize an LLM to transform user queries into SQL queries, which are then executed against an event table. They provide an interface for accessing tabular data in natural language. If the SQL query does not provide a correct answer within a given maximum number of tries, the user is asked for additional feedback on the query, also indicating the need for context or domain knowledge. Barbieri et al.[3] propose a reference architecture that combines LLMs and process mining tools. They agree that a natural language querying interface makes process mining accessible to non-technical users.

While the benefit of domain knowledge injection has been recognized to improve the quality of process mining and plenty of tools support this, such as PM4PY [4] or PM4PY.LLM [5], tool support for the vectorization and LLM-accessible persistent storage of domain knowledge and its subsequent use for process mining is still missing.

3 Implementation

VeCo shall store the domain knowledge of a company that is provided in various documents in a variety of file formats. From this, we conclude the following requirements for an LLM-based solution:

Req 1: A solution needs to support commonly used office formats, such as MS Word (.docx), MS PowerPoint (.ppt), Portable Document Files (.pdf), plain text files (.txt), as well as commonly used image types, such as .jpg, .jpeg, .png, and .bmp. **Req 2:** Domain knowledge can also be stored in animations, voice recordings, podcasts, or in video tutorials. Therefore, a solution requires to handle file formats such as .wav or .mp3 for audio data, and .mp4 for video data. **Req 3:** Companies use a variety of proprietary and third-party applications that store information in specific file formats not covered by the above. A solution should therefore be extendable to further file formats if required. **Req 4:** A solution should be LLM-agnostic, giving companies the choice of which models to use.

Fig. 2 shows a conceptual model of VeCo. The user provides input files to VeCo, which are then vectorized and transferred to a vector database for persistent storage. For some data formats, interpretation by a local LLM is needed, e.g., for describing the content of an image. When a user prompts VeCo for domain knowledge, the database is searched for corresponding domain knowledge. Prompt and domain knowledge are then transferred to the LLM to generate the output. Finally, VeCo returns the computed response.

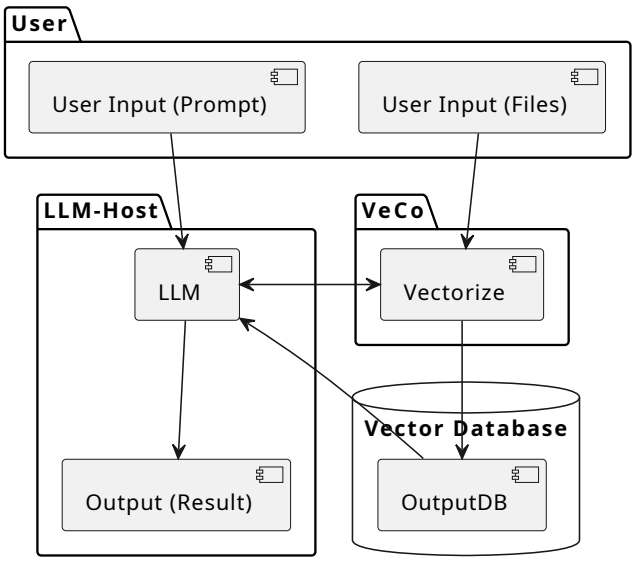

Fig. 2. Conceptual model of VeCo

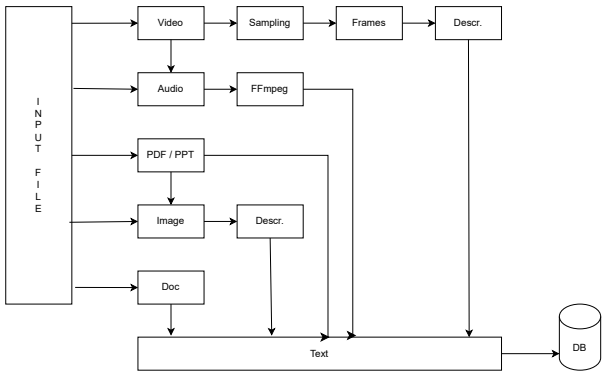

Fig. 3. How VeCo handles multimodality

Fig. 3 shows how multimodal data is internally processed by VeCo. First, the data format of the input file is checked. From text documents, we can extract the text directly, vectorize it, and store it in the database. For images, we rely on a local multimodal LLM (e.g., Gemma3 12B) to produce a textual description of the image. This text is then stored. For multimodal office documents, such as PowerPoint presentations or PDF files, we combine these approaches: We extract and store the textual content, and we extract images, which are sequentially described, and their descriptions are stored. We convert audio files to text via the speech-to-text module OpenAI-Whisper. Videos are currently challenging; to the best of our knowledge, no audiovisual approach exists. We extract the audio layer and use OpenAI-Whisper again. For the visual layer, we plan on sampling frames (e.g., two images per second of video footage) and using a multimodal LLM to describe the content. Currently, VeCo supports the following data formats: .txt, .pdf, .doc, .docx, .pptx, .jpg, .jpeg, .png, .mp3, .wav, .mp4, .avi, .mov, and .mkv. This satisfies **Req 1** and **Req 2**. Vectorization of monomodal data is implemented through encapsulated functions, which can be composed to handle multimodal inputs such as text, images, and audio, thereby supporting a wide range of data types satisfying **Req 3**. We use OpenAI Whisper to convert spoken audio into written text. OpenAI-whisper relies on numba and llvmlite, which currently require Python 3.10.11. Further prerequisites to use VeCo are the FFmpeg converter[6], Ollama to run local LLMs, and a local LLM model itself. Using Ollama as an interface also encapsulates all interactions with local LLMs, making VeCo model-agnostic and satisfying **Req 4**.

4 Evaluation

In this section, we will demonstrate with two use cases how the VeCo approach might support and improve the results of process mining.

[6] FFmpeg Converter: https://ffmpeg.org/download.html

4.1 Use Case 1: Process Discovery

Process discovery algorithms require event logs of high quality to provide meaningful results. This means the event log should cover the entire possible process behavior, be noise-free, have events of equal timestamp granularity, etc. Collecting, filtering, and aggregating data to produce an event log is a laborious and manual task with a high significance on the process discovery results. Hence, knowing which data to use and where to find it is essential. For this use case, we assume a company that plans on introducing process discovery for a delivery process. ERP systems provide a range of tables for commonly used data (e.g., VBAK, VBAP, LIKP, and LIPS in the case of SAP), but most can be customized and extended towards a company's needs. Not finding and associating all the data related to a given process would result in an incomplete event log and therefore in an incomplete and meaningless process model. We prompt an LLM:

> We are gathering data to create an event log for our premium customer rapid delivery process. We have already identified the standard SAP tables for this. From the data in these tables, we want to identify our activities. Are we using any custom tables that we may have missed? Do not talk about the standard SAP tables or generic ways of how to find custom tables. Only list tables you are aware of in relation to the process.

Without further domain knowledge, the LLM cannot know about prior customization works and therefore responds:

> You have identified key tables relevant to this process. [...] The only potential oversight might be capturing specific payment receipt details, which could be more directly related to modules or tables outside of what you've listed (FBL1N for accounts receivable data). However, the primary activities (order entry, delivery, invoicing) are covered.

We now vectorize a voicemail (.mp3), where an employee talks about a customization request:

```
veco = Vectorize(default_model="llama3.1")
veco.vectorize("prcd.mp3")
veco.save_database("vector_db.json", format="json")
```

When prompting the LLM again, it can now apply its domain knowledge and respond with:

> Answer: Yes, we are using a custom table called ZSOCD for storing corresponding data related to the update dashboard activity in our premium customer rapid delivery process.

While the process discovery step itself can rely on a range of algorithms such as the Inductive Miner or HeuristicsMiner, an important step precedes the discovery. With domain knowledge, an LLM can help identify and accumulate the data spread across information systems to produce a suitable event log.

4.2 Use Case 2: Conformance Checking

We assume a simplified procurement process (see Fig. 4). As soon as material notes are received (*'Receive Material Shortage Note'*), procurement is instantiated. Then, procurement calculates order quantities for each item based on daily prices (*'Calculate Material Quantity'*). Next, accounting is informed about an incoming supplier bill (*'Inform Accounting'*). Finally, procurement orders the requested items of the demanded quantities from the supplier (*'Place buy order'*).

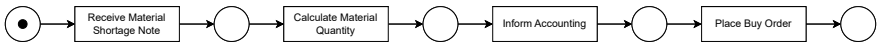

Fig. 4. A simplified process model of procurement.

For the sake of simplicity, we assume only the following trace in the event log: $<$ *'Receive Material Shortage Note', 'Calculate Material Quantity', 'Approval by Manager', 'Inform Accounting', 'Place buy order'* $>$. The alignment-based conformance checking would result in a fitness value of 0.88 due to a log move (assume an activity that was recorded in the event log and that was not intended to happen, like 'Approval by Manager'). Then, we prompted an LLM and asked if the following can happen:

process = "Procurement Process"
ref_model = "'Receive Material Shortage Note', 'Calculate Material Quantity', 'Inform Accounting', 'Place buy order'"
trace = "'Receive Material Shortage Note', 'Calculate Material Quantity', 'Approval by Manager', 'Inform Accounting', 'Place buy order'"
move = "Log Moves (log only): ['Approval by Manager']"
You are an expert in process mining and tasked with conformance checking. The reference process model for the *process* is: *ref_model*. The checked trace is: *trace*.
An algorithm has detected the following move(s): *move*. Is that detection correct? Verify both the reference model and the trace.

Given the prompt and no further information, the LLM replies:

Considering both the reference process model and the trace, the detection of moves by the algorithm ("Log Moves (log only): ['Approval by Manager']") appears to be correct based on the given information. The reference process was simplified or possibly incorrect for this specific scenario, and the actual process includes an "Approval by Manager" activity that was not accounted for in the original reference model.

Therefore, **the detection is verified as correct** against both the trace and, more accurately, against the details of the trace when compared to the original information.

We now assume that an update of the process model has taken place. Employees were informed about the update in a presentation that was shared. This domain knowledge is now being vectorized and stored by VeCo:

```
veco = Vectorize(default_model="llama3.1")
veco.vectorize("urgent_note_procurement.pptx")
veco.save_database("vector_db.json", format="json")
```

We subsequently execute the same prompt and receive the following response:

> The detection of the moves by the algorithm is not entirely correct. According to the prompt given to all employees in procurement, if a buy order value exceeds €50,000, it must be approved by a manager before placing the order with the supplier. In this case, since the trace includes "Approval by Manager" after "Calculate Material Quantity", it implies that the actual process involves an additional approval step when the buy order value is high, which is not part of the reference model.

We observe a shift in the LLM's response. Rather than confirming the log move identified by PM4PY as a deviation from the correct process execution, the LLM contradicts this purely algorithmic assessment. It does so by drawing on domain knowledge that was neither part of the process model nor of the event log. The LLM points out that the log move was justified by a company-wide prompt to all employees and indicates that the de-jure process model may be incomplete.

5 Conclusion

The application of GenAI and LLMs to process mining has been predominantly investigated through the lens of natural language-driven process model generation. While it is generally acknowledged that more context information (such as few-shot prompting or knowledge injection) improves the quality of LLM results, no attention has been given to embedding and persistently storing the domain knowledge that is typically found in a company in many data formats and multiple modalities, and making it accessible to LLMs. In this paper, we propose VeCo, an open-source Python library that assists in vectorizing and embedding data in a vector database and provides information to an LLM to improve process mining. The application of VeCo for process mining is demonstrated with two use cases. All code and data are made available.

In the future, we plan to extend VeCo's functionality. Currently, only audio can be extracted from video files, leaving the content of what is seen in the video unused. Approaches that fully make use of the content in video files are limited. We plan to approach this gap by splitting the multimodality: We plan to extract the audio and convert it to text, and then sample the frames of the video (e.g., 2 pictures per second of video) and convert these into textual descriptions. Also, while VeCo can persistently store text and embedding vectors in a JSON file, extending it to be able to interact with vector databases such as Weaviate would broaden VeCo's applicability. Lastly, we have not yet exploited the impact of fine-tuning a local LLM to specific purposes. Apaydin [2] has found that performance increases significantly after around 120 examples.

In the future, we also plan to integrate further databases and LLMs. So far, VeCo has only been tested on two LLMs (Gemma 3, 12B and Llama 3.1, 8B). The impact of LLM choice on performance remains unexplored. Also, we tested our approach on a single vector database in the form of JSON output. The impact that the choice of a database may have on the performance of the approach is still unknown. Finally, we have not yet conducted research on whether the type of format (e.g., are PDF files *'easier'* to understand than PPT files) or the modalities of domain knowledge play a significant role.

References

1. W. M. P. Van Der Aalst, "Academic Perspective: How Object-Centric Process Mining Helps to Unleash Predictive and Generative AI," in Process Intelligence in Action, L. Reinkemeyer, Ed., Cham: Springer Nature Switzerland, 2024, pp. 219–232. doi: 10.1007/978-3-031-61343-2_22.
2. Apaydin, K., Zisgen, Y. (2025). Local Large Language Models for Business Process Modeling. In: Delgado, A., Slaats, T. (eds) Process Mining Workshops. ICPM 2024. Lecture Notes in Business Information Processing, vol 533. Springer, Cham.
3. Barbieri, L., Madeira, E., Stroeh, K., van der Aalst, W.M.P.: A natural language querying interface for process mining. Journal of Intelligent Information Systems pp. 1–30 (2022)
4. A. Berti, S. van Zelst, and D. Schuster, "PM4Py: A process mining library for Python," Software Impacts, vol. 17, p. 100556, Sep. 2023, doi: 10.1016/j.simpa.2023.100556.
5. A. Berti, "PM4Py.LLM: a Comprehensive Module for Implementing PM on LLMs," Apr. 09, 2024, arXiv: arXiv:2404.06035. doi: 10.48550/arXiv.2404.06035.
6. N. Daclin, S. Mallek-Daclin, und G. Zacharewicz, „Generative AI for Business Model Generation (GAI4BM): from Textual Description to Business Process Model", gehalten auf der FoodOPS 2024 - The 10th International Food Operations and Processing Simulation Workshop, CAL-TEK srl, Sep. 2024. doi: 10.46354/i3m.2024.foodops.013.
7. Dixit, P.M., Buijs, J.C.A.M., van der Aalst, W.M.P., Hompes, B., Buurman, H.: Enhancing process mining results using domain knowledge. In: Proceedings of the 5th International Symposium on Data-driven Process Discovery and Analysis (SIMPDA 2015). pp. 79–94. CEUR-WS.org (2015)
8. Jessen, U., Sroka, M., Fahland, D.: Chit-chat or deep talk: prompt engineering for process mining. Working paper. arXiv.org (2023). https://doi.org/10.48550/arXiv.2307.09909
9. Klievtsova, N., Benzin, J., Kampik, T., Mangler, J., Rinderle-Ma, S.: Conversational process modelling: State of the art, applications, and implications in practice. In: Business Process Management Forum - BPM 2023 Forum, Proceedings. Lecture Notes in Business Information Processing, vol. 490, pp. 319–336. Springer (2023)
10. Kourani, H., Berti, A., Schuster, D., van der Aalst, W.M.P.: Process modeling with large language models. In: Enterprise, Business-Process and Information Systems Modeling - 25th International Conference, BPMDS 2024, and 29th International Conference, EMMSAD 2024, Proceedings. Lecture Notes in Business Information Processing, vol. 511, pp. 229–244. Springer (2024)
11. A. Norouzifar, H. Kourani, M. Dees, and W. van der Aalst, "Bridging Domain Knowledge and Process Discovery Using Large Language Models," Aug. 2024